

Projektrapport: Signalbehandling i Python

Valfri fördjupning inom Python för Data Science

Kurs: Fördjupning i Pythonprogrammering

Datum: September 2026

Ämne: Signalbehandling & Machine Learning

Inlämning: Skriftlig rapport & GitHub

1. SYFTE OCH VAL AV OMRÅDE

Syftet med detta projekt är att självständigt fördjupa mig i hur signalbehandling kan användas inom Data Science. Projektet fokuserar på att förbehandla ljuddata (.wav-signaler) och använda frekvensinformation för att träna en enkel maskininlärningsmodell för tillståndsbaserat underhåll (Predictive Maintenance).

Området är valt eftersom det är direkt relevant för min kommande LIA-plats där företaget arbetar med analys av maskindata och signalbehandling.

Avgränsning (efter lärarens feedback):

För att hålla projektet välavgränsat och genomförbart har jag fokuserat enbart på **ljusignaler** (ljud från maskiner) och valt enbart **Fast Fourier Transform (FFT)** via `scipy.fft` för feature extraction. Mer komplexa metoder som MFCC eller realtidsfiltrering har medvetet valts bort.

2. METOD OCH TEKNISK BESKRIVNING

Projektets kod är uppbyggd som en enkel Python-pipeline i tre steg:

- Datagenerering:** Skapar korta sinusvågor med brus i tidsdomänen för att simulera normala maskinljud (låg frekvens, 50 Hz) och felaktiga maskinljud (hög frekvens, 400 Hz).
- Signalbehandling (FFT):** Med hjälp av `scipy.fft` omvandlas ljudsignalen från tidsdomänen till frekvensdomänen. Omvandlingen gör det möjligt att analysera vilka frekvenser som dominerar i ljudet.
- Feature Extraction & ML:** Tre statistiska drag beräknas från spektromet (medelvärde, maxvärde och standardavvikelse). En `RandomForestClassifier` från `scikit-learn` tränas därefter på dessa värden för att klassificera ljudet.

3. BEROENDEN

Projektet kräver följande Python-bibliotek (angivna i `requirements.txt`):

- `numpy` – För numeriska beräkningar och signalgenerering.
- `scipy` – För Fast Fourier Transform (`scipy.fft`).
- `librosa` & `soundfile` – För hantering av ljuddata.
- `scikit-learn` – För uppdelning av data och träning av Random Forest-modellen.

4. KÄLLKOD (MAIN.PY)

```
import numpy as np
import librosa
from scipy.fft import fft
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Funktion för att extrahera features med FFT
def extract_fft_features(signal, sr):
    n = len(signal)
    fft_spectrum = np.abs(fft(signal))[:n // 2]
    mean_freq = np.mean(fft_spectrum)
    max_freq = np.max(fft_spectrum)
    std_freq = np.std(fft_spectrum)
    return [mean_freq, max_freq, std_freq]

# 2. Skapa dataset med syntetiska ljudsignaler
X, y = [], []
sample_rate = 22050
duration = 1.0
t = np.linspace(0, duration, int(sample_rate * duration))

# Normala ljud (50 Hz)
for _ in range(10):
    noise = np.random.normal(0, 0.05, len(t))
    signal = np.sin(2 * np.pi * 50 * t) + noise
    X.append(extract_fft_features(signal, sample_rate))
    y.append(0)

# Felaktiga ljud (400 Hz)
for _ in range(10):
    noise = np.random.normal(0, 0.05, len(t))
    signal = np.sin(2 * np.pi * 400 * t) + noise
    X.append(extract_fft_features(signal, sample_rate))
    y.append(1)

X, y = np.array(X), np.array(y)

# 3. Träna och utvärdera modell
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = RandomForestClassifier(n_estimators=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Träning klar! Modellens pricksäkerhet: {accuracy_score(y_test, y_pred) * 100:.0f}%")
```

5. RESULTAT OCH SJÄLVREFLEKTION

Resultat: Skriptet körs utan fel och modellen når hög pricksäkerhet. Genom att omvandla ljudet till frekvensplanet blir det enkelt för Random Forest-modellen att skilja på normala och felaktiga maskinljud.

Begränsningar: Projektet täcker enbart korta, syntetiska ljudfiler i `.wav`-format. Det innehåller inte MFCC-analys, avancerad filtrering eller realtidshantering.

Lärdomar: Att arbeta med FFT gav mig en god förståelse för hur viktig förbehandling och feature extraction är inom Data Science. Istället för att skicka in råa tidsserier direkt till en modell underlättar FFT klassificeringen avsevärt.

6. STARTKÄLLOR OCH DOKUMENTATION

- **SciPy Documentation:** `scipy.fft` – Modulen för Fast Fourier Transform.
- **Librosa Documentation:** Audio Feature Extraction i Python.
- **Scikit-learn Documentation:** `RandomForestClassifier` och utvärderingsmått.